

Bangladesh University of Engineering & Technology



Course No: EEE 212

Course Title: Numerical Technique Laboratory

Name of The Project: Solve the Poisson's Equation of electrostatics using numerical method for 2-dimensional space

Submitted to: Sadman Sakib Ahbab

(Lecturer, EEE, BUET)

Dibakor Roy

(TA, EEE, BUET)

Date of Submission: 11/02/2023

Name: Fariha Noor

Tomalika Mazumdar

Student ID: 2006078

2006079

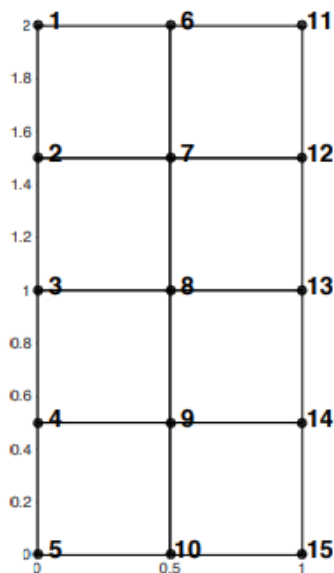
Group Name: Megami

Objective:

To solve Poisson Equation for electrostatic potential which is an elliptic partial differential equation, in MATLAB.

Method for Solving the Poisson Equation:

Geometrically a 2-D grid is naturally linked to a matrix. When forming the matrix equation, we need to use a linear indexing to transfer this 2-D grid function to a 1-D vector function. We can skip this artificial linear indexing and treat our function $u(x, y)$ as a matrix function $u(i, j)$. The multiple subscript indexing to the linear indexing is build into the matrix. The matrix is still stored as a 1-D array in memory. The default linear indexing in MATLAB is column wise. For example, a matrix $A = \begin{bmatrix} 2 & 9 & 4 \\ 3 & 5 & 11 \end{bmatrix}$ is stored in memory as the array $[2 \ 3 \ 9 \ 5 \ 4 \ 11]'$. One can use a single index to access an element of the matrix, e.g., $A(4) = 5$. In MATLAB, there are two matrix systems to represent a two dimensional grid: the geometry consistent matrix and the coordinate consistent matrix. To fix ideas, we use the following example. The domain $(0, 1) \times (0, 2)$ is decomposed into a uniform grid with mesh size $h = 0.5$. The linear indexing of these two systems are illustrate in the following figures.



(a) meshgrid

The command `[x,y] = meshgrid(0:0.5:1,2:-0.5:0)` will produce 5×3 matrices. Note that the flip of the ordering `2:-0.5:0` in the y-coordinate makes the matrix is geometrically consistent with the domain in the sense that the shape of the matrix matches the shape of the domain. This index system is illustrated in Fig. 1(a).

```

1  >> [x,y] = meshgrid(0:0.5:1,2:-0.5:0)
2  x =
3      0      0.5000      1.0000
4      0      0.5000      1.0000
5      0      0.5000      1.0000
6      0      0.5000      1.0000
7      0      0.5000      1.0000
8  y =
9      2.0000      2.0000      2.0000
10     1.5000      1.5000      1.5000
11     1.0000      1.0000      1.0000
12     0.5000      0.5000      0.5000
13      0          0          0

```

In this geometrically consistent system, however, there is an inconsistency of the convention of notation of matrix and Cartesian coordinate. Let us figure out the mapping between the algebraic index (i,j) and the geometric coordinate (x_i, y_j) of a grid point. In the command $[x,y] = \text{meshgrid}(x_{\min}:h_x:x_{\max}, y_{\max}:h_y:y_{\min})$, the coordinate of the (i, j) -th grid point is $(x_j, y_i) = (x_{\min} + (j - 1)h_x, y_{\min} + (n - i + 1)h_y)$, which violates the convention of associating index i to x_i and j to y_j . For a matrix entry $A(i,j)$, the 1st index i is the row and the 2nd j is the column while in Cartesian coordinate, i is usually associated to the x -coordinate and j to the y -coordinate.

The commands `sub2ind` and `ind2sub` is designed for such purpose. We include two examples below and refer to the documentation of MATLAB for more comprehensive explanation and examples. The command `k=sub2ind([3 5],2,4)` will give `k=11` and

`[i,j]=ind2sub([3 5],11)` produces `i=2, j=4`. In the input `sub2ind(size, i,j)`,

the `i,j` can be arrays of the same dimension. In the input `ind2sub(size, k)`, the `k` can

be a vector and the output `[i,j]` will be two arrays of the same length of `k`. Namely these two commands support vectors arguments.

For a matrix function $u(i,j)$, $u(:)$ will change it to a 1-D array using the column-wise

linear indexing and `reshape(u,m,n)` will change a 1-D array to a 2-D matrix function.

A more intuitive way to transfer multiple subscripts into the linear index is to explicitly store an index matrix. For meshgrid system, use

`idxmat = reshape(uint32(1:m*n), m, n)` Then one can easily get the linear indexing of the j -th column of a $m \times n$ matrix by using `idxmat(:,j)` which is equivalent to `sub2ind([m n], 1:m, j*ones(1,m))` but much easier and intuitive. The price to pay is the extra memory for the full matrix `idxmat` which can be minimized using `uint32`.

Similarly we can generate matrices to store the subscripts. For the `meshgrid` system

```

1  >> [jj,ii] = meshgrid(1:3,1:5)
2  jj =
3      1      2      3
4      1      2      3
5      1      2      3
6      1      2      3
7      1      2      3
8  ii =
9      1      1      1
10     2      2      2
11     3      3      3
12     4      4      4
13     5      5      5

```

Then `ii(k),jj(k)` will give the subscript of the k -th node. Last we discuss the access of boundary points which is important when imposing boundary conditions. Using subscripts of `meshgrid` system, the index of each part of the boundary of the domain is `meshgrid`: left - `(:,1)` right - `(:,end)` top - `(1,:)` bottom - `(end,:)`

the coordinate consistency: i to x and j to y . Thus the left boundary will be $i = 1$ corresponding to $x = x_1$. The linear indexes of all boundary nodes can be found by the following codes

```

1  isbd = true(size(u));
2  isbd(2:end-1,2:end-1) = false;
3  bdidx = find(isbd(:));

```

Code Explanation:

The Poisson equation relates the electric potential V to the charge density ρ in a given region of space:

$$\nabla^2 V = -\rho/\epsilon_0$$

where ϵ_0 is the vacuum permittivity.

Here is a step-by-step explanation of the code:

The first line clears the command window and any previous figures.

The input parameters are entered by the user via the command window. These include the number of charges, the minimum and maximum x and y coordinates, and the electric potential values at the top, bottom, left and right boundaries.

The code sets up a grid of points with a spacing of dx and dy in the x and y directions, respectively, using the minimum and maximum x and y coordinates entered by the user.

The charge distribution ρ is calculated by summing over all the charges entered by the user. Each charge is modeled as a Gaussian distribution with a given magnitude, width, and location. The Gaussian distribution is multiplied by the normalization factor $1/(2\pi\sigma^2)$.

The code sets up matrices A and b for the numerical solution of the Poisson equation. The matrix A represents the coefficients of the linear system of equations, while b represents the right-hand side.

The interior points of the grid are then used to form the linear system of equations. This is done by iterating over all the interior points (excluding boundary points) and applying the finite difference method to obtain the coefficients of the linear system of equations.

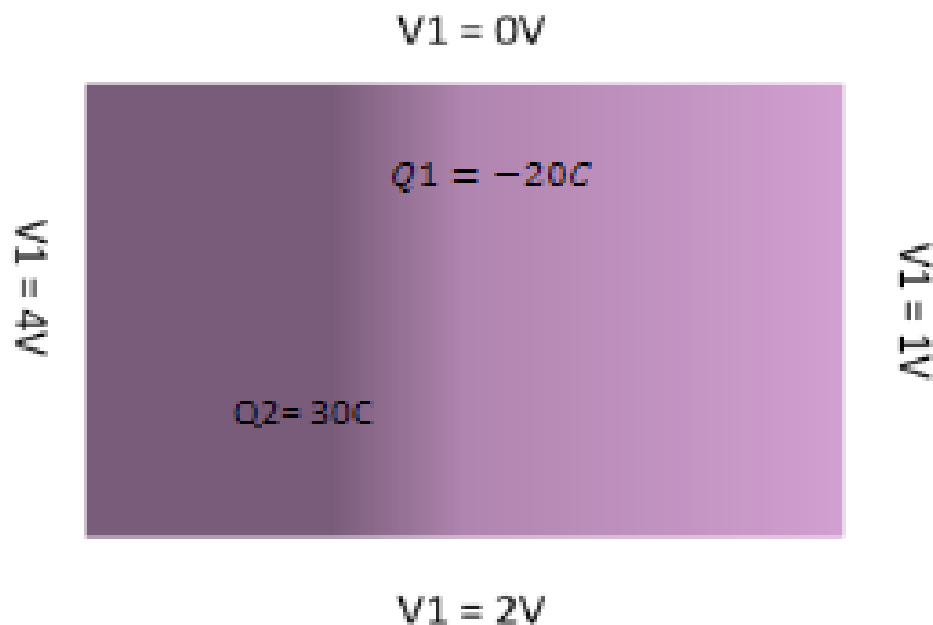
The boundary conditions are then defined by setting the values of V at the edges of the grid to the values entered by the user. This is done by iterating over the boundary points and setting the corresponding rows and columns of the matrices A and b to the appropriate values.

The code solves the linear system of equations for V using the matrices A and b. The solution is stored as a column vector.

The solution V is then reshaped into a 2D grid with dimensions $N_y \times N_x$.

Finally, the electric potential is plotted as a contour plot using the contourf function in MATLAB. The resulting plot shows the distribution of the electric potential across the grid.

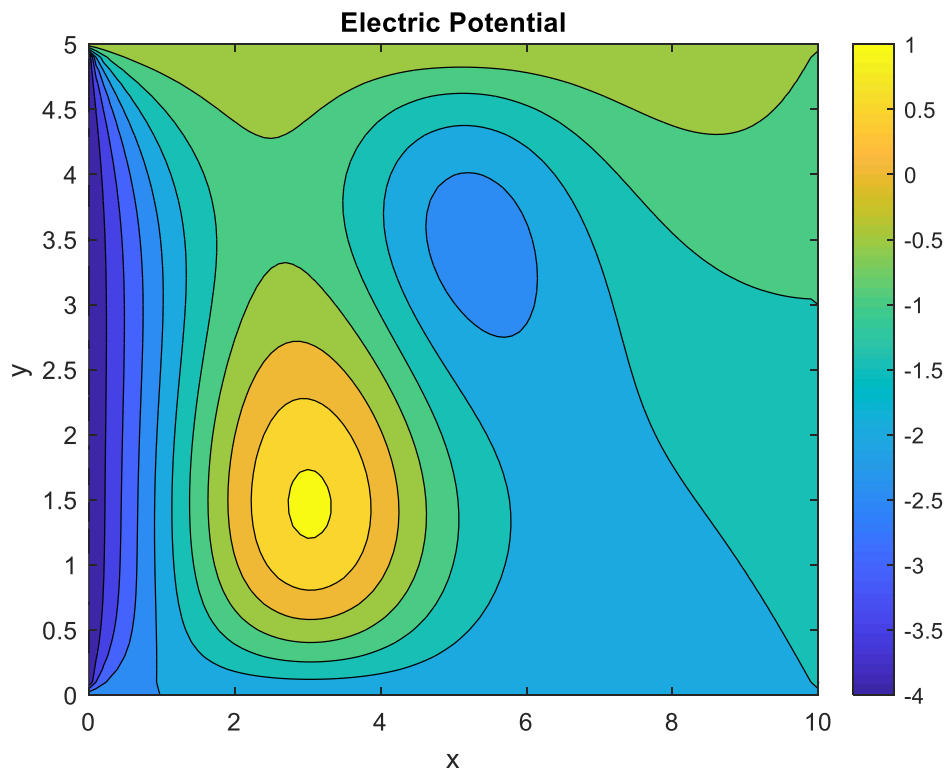
Example Problem:



Assuming the location of the charges we can solve the example:

Command Window

```
Enter the number of charges: 2
Enter the minimum x-coordinate: 0
Enter the maximum x-coordinate: 10
Enter the minimum y-coordinate: 0
Enter the maximum y-coordinate: 5
Enter charge 1 value: -20
Enter charge 1 x-coordinate: 5
Enter charge 1 y-coordinate: 4
Enter charge 2 value: 30
Enter charge 2 x-coordinate: 3
Enter charge 2 y-coordinate: 1
Enter electric potential at the top boundary: 0
Enter electric potential at the bottom boundary: 2
Enter electric potential at the left boundary: 4
Enter electric potential at the right boundary: 1
```

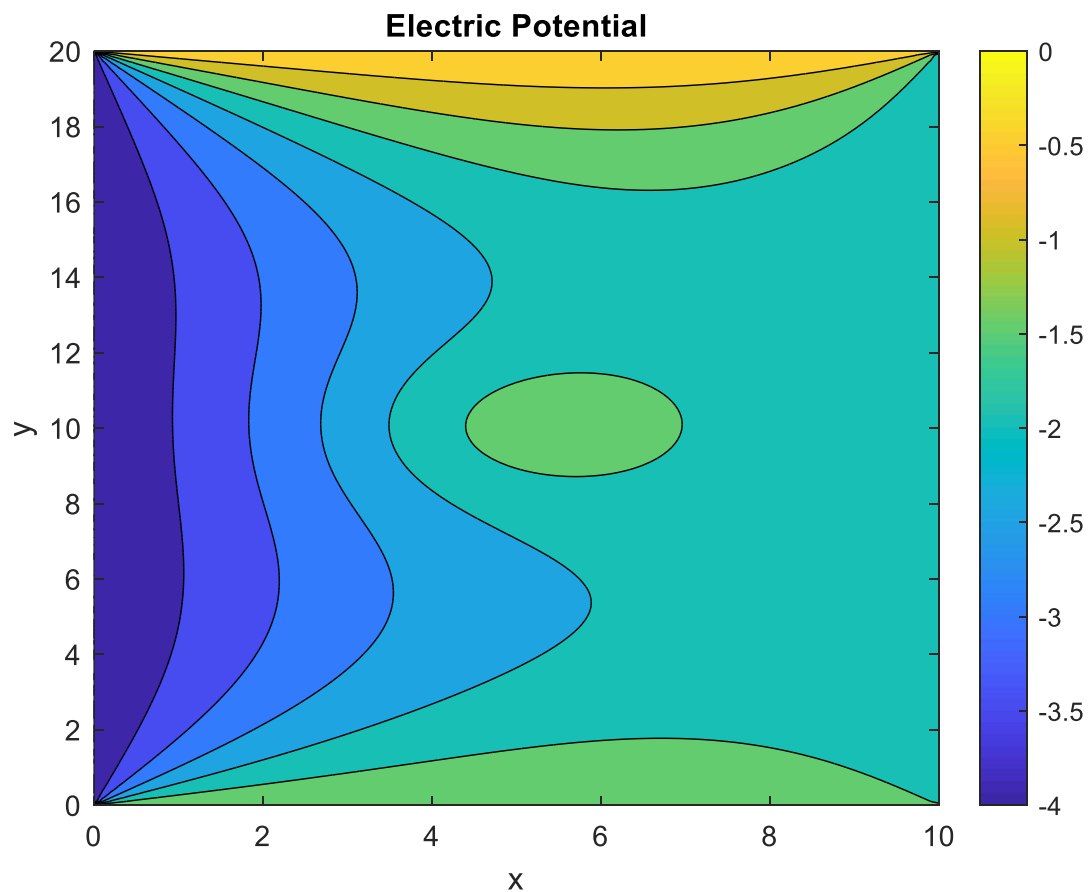


Now some test cases solved with the code:

Case 1:

Case of a single positive point charge located at the center of the rectangular space with non zero electric potential at the boundaries.

```
Enter the number of charges: 1
Enter the minimum x-coordinate: 0
Enter the maximum x-coordinate: 10
Enter the minimum y-coordinate: 0
Enter the maximum y-coordinate: 20
Enter charge 1 value: 5
Enter charge 1 x-coordinate: 5
Enter charge 1 y-coordinate: 10
Enter electric potential at the top boundary: 0
Enter electric potential at the bottom boundary: 1
Enter electric potential at the left boundary: 4
Enter electric potential at the right boundary: 2
```

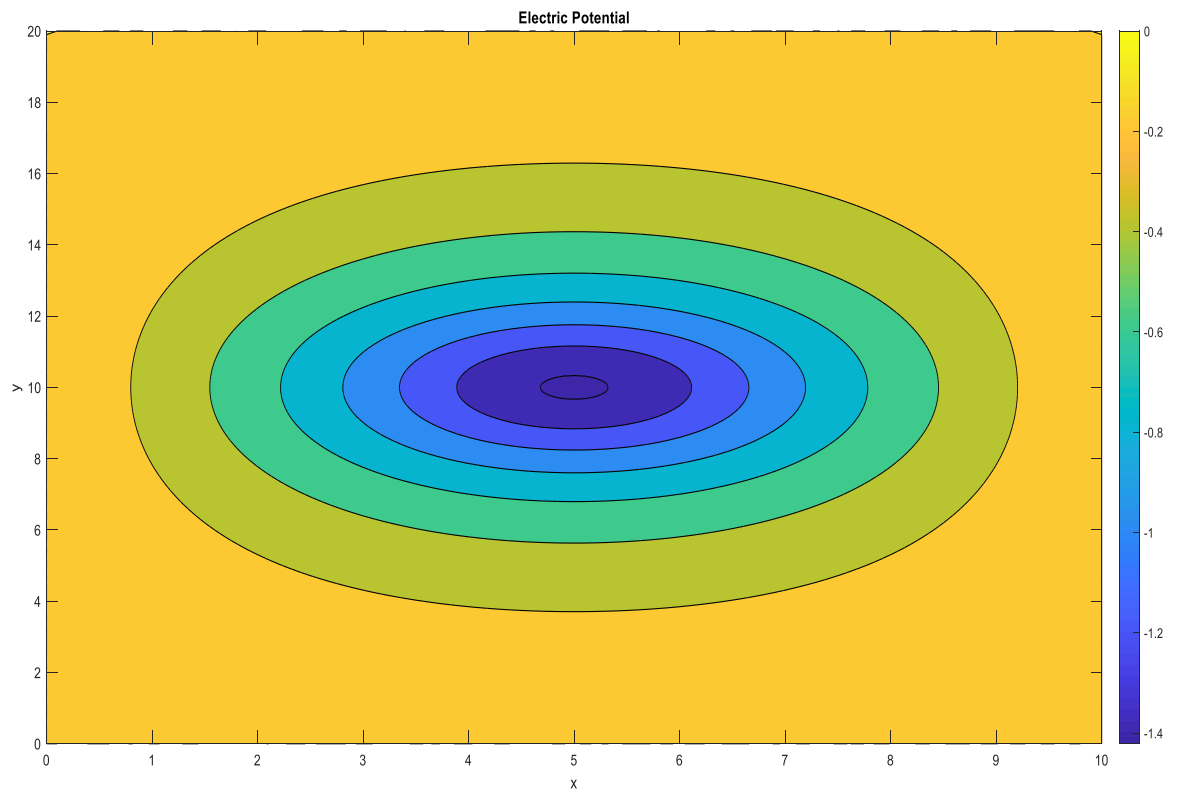


Case 2:

Case of a single negative point charge located at the center of the rectangular space with zero electric potential at the boundaries.

Command Window

```
Enter the number of charges: 1  
Enter the minimum x-coordinate: 0  
Enter the maximum x-coordinate: 10  
Enter the minimum y-coordinate: 0  
Enter the maximum y-coordinate: 20  
Enter charge 1 value: -5  
Enter charge 1 x-coordinate: 5  
Enter charge 1 y-coordinate: 10  
Enter electric potential at the top boundary: 0  
Enter electric potential at the bottom boundary: 0  
Enter electric potential at the left boundary: 0  
Enter electric potential at the right boundary: 0  
fx >> |
```

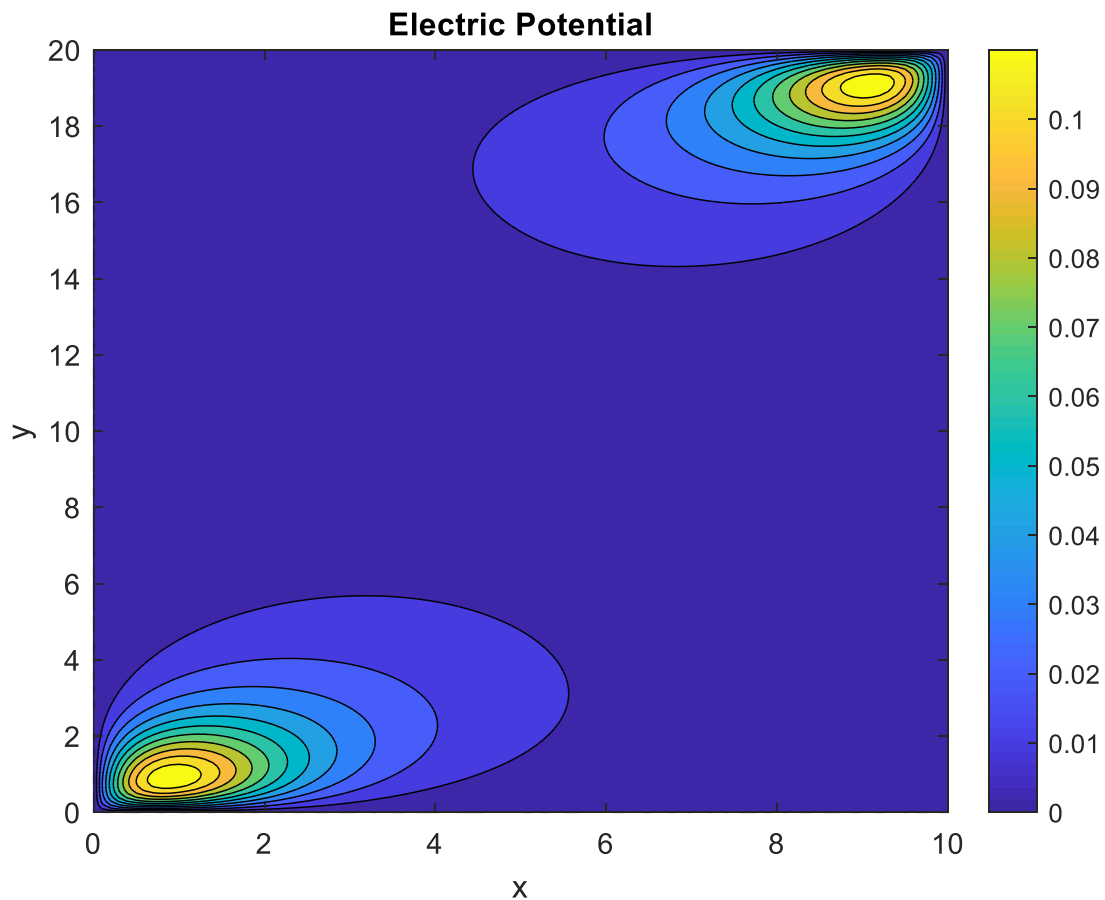


Case 3:

Case of two positive point charges of equal magnitude located at opposite corners of the rectangular space with zero electric potential at the boundaries.

Command Window

```
Enter the number of charges: 2
Enter the minimum x-coordinate: 0
Enter the maximum x-coordinate: 10
Enter the minimum y-coordinate: 0
Enter the maximum y-coordinate: 20
Enter charge 1 value: 5
Enter charge 1 x-coordinate: 0
Enter charge 1 y-coordinate: 0
Enter charge 2 value: 5
Enter charge 2 x-coordinate: 10
Enter charge 2 y-coordinate: 20
Enter electric potential at the top boundary: 0
Enter electric potential at the bottom boundary: 0
Enter electric potential at the left boundary: 0
Enter electric potential at the right boundary: 0
```

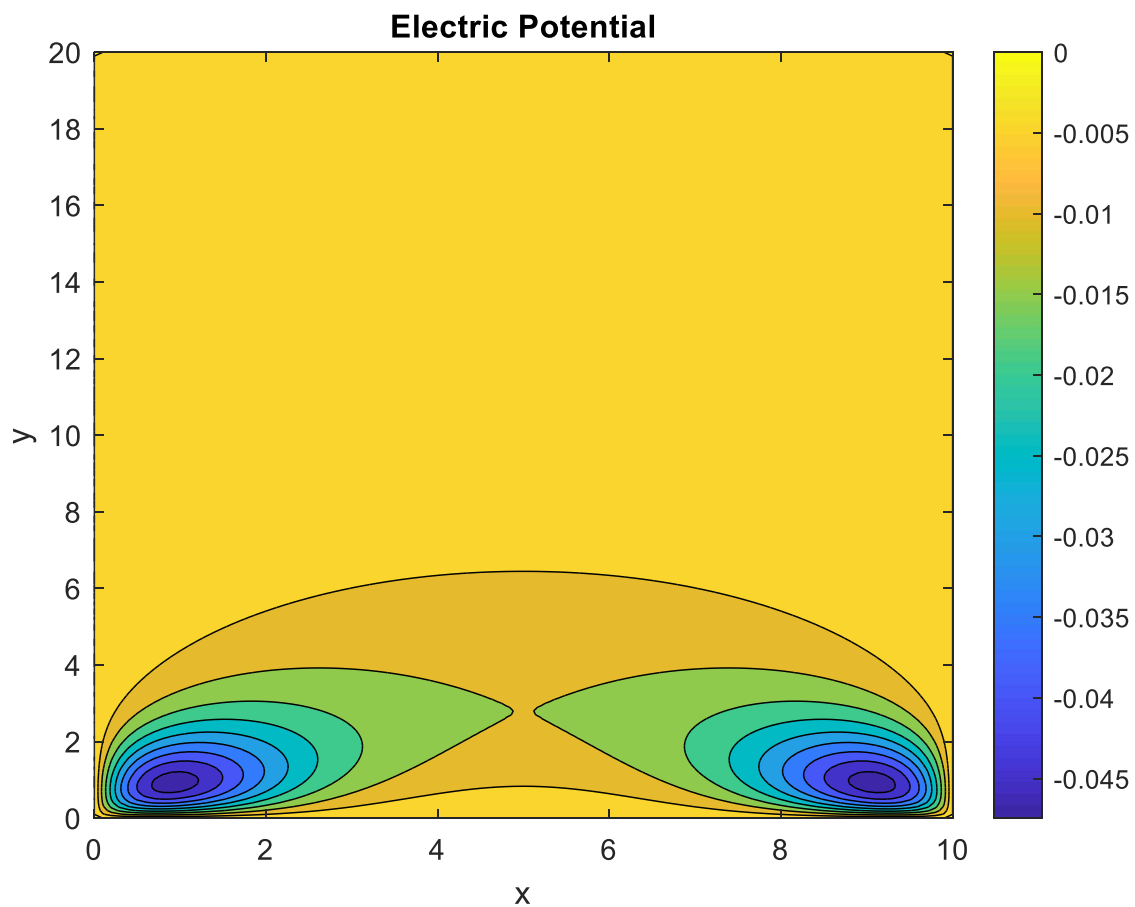


Case 4:

Case of two negative point charges of equal magnitude located at consecutive corners of the rectangular space with zero electric potential at the boundaries.

Command Window

```
Enter the number of charges: 2  
Enter the minimum x-coordinate: 0  
Enter the maximum x-coordinate: 10  
Enter the minimum y-coordinate: 0  
Enter the maximum y-coordinate: 20  
Enter charge 1 value: -2  
Enter charge 1 x-coordinate: 0  
Enter charge 1 y-coordinate: 0  
Enter charge 2 value: -2  
Enter charge 2 x-coordinate: 10  
Enter charge 2 y-coordinate: 0  
Enter electric potential at the top boundary: 0  
Enter electric potential at the bottom boundary: 0  
Enter electric potential at the left boundary: 0  
Enter electric potential at the right boundary: 0
```

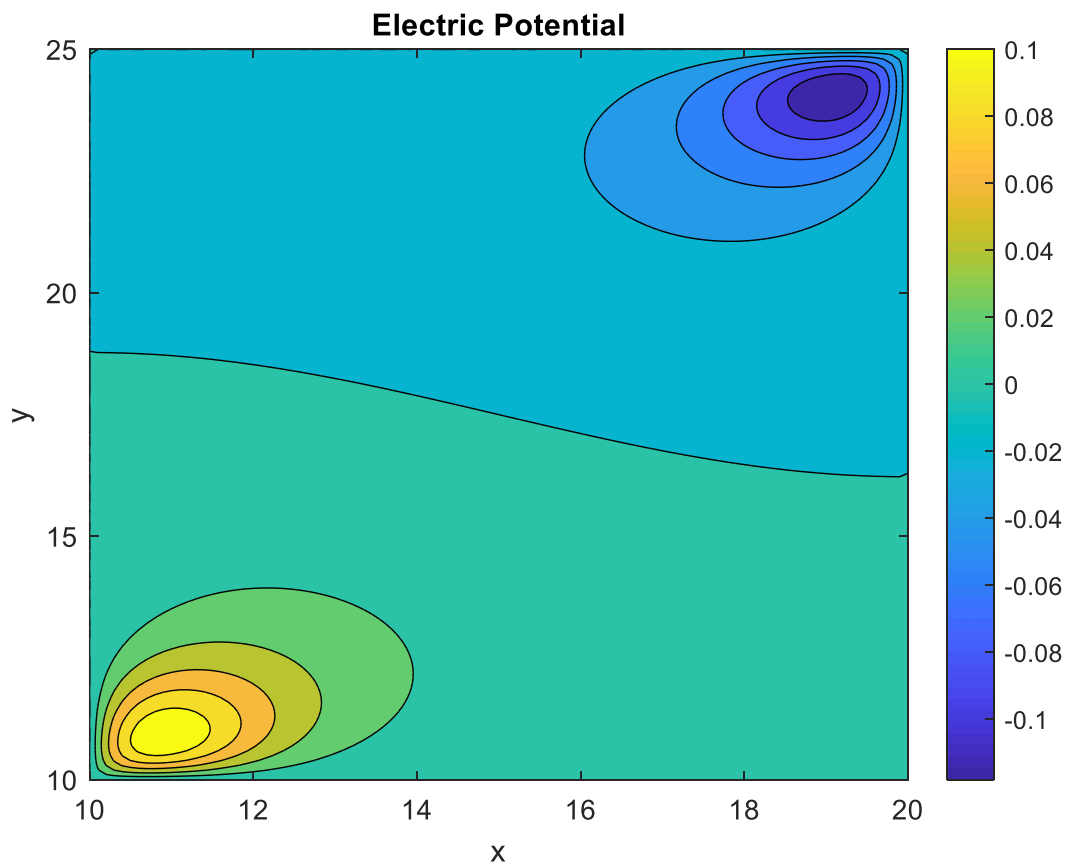


Case 5:

Case of two point charges of equal magnitude and opposite signs located at opposite corners of the rectangular space with zero electric potential at the boundaries.

Command Window

```
Enter the number of charges: 2
Enter the minimum x-coordinate: 10
Enter the maximum x-coordinate: 20
Enter the minimum y-coordinate: 10
Enter the maximum y-coordinate: 25
Enter charge 1 magnitude: 5
Enter charge 1 x-coordinate: 10
Enter charge 1 y-coordinate: 10
Enter charge 2 magnitude: -5
Enter charge 2 x-coordinate: 20
Enter charge 2 y-coordinate: 25
Enter electric potential at the top boundary: 0
Enter electric potential at the bottom boundary: 0
Enter electric potential at the left boundary: 0
Enter electric potential at the right boundary: 0
```

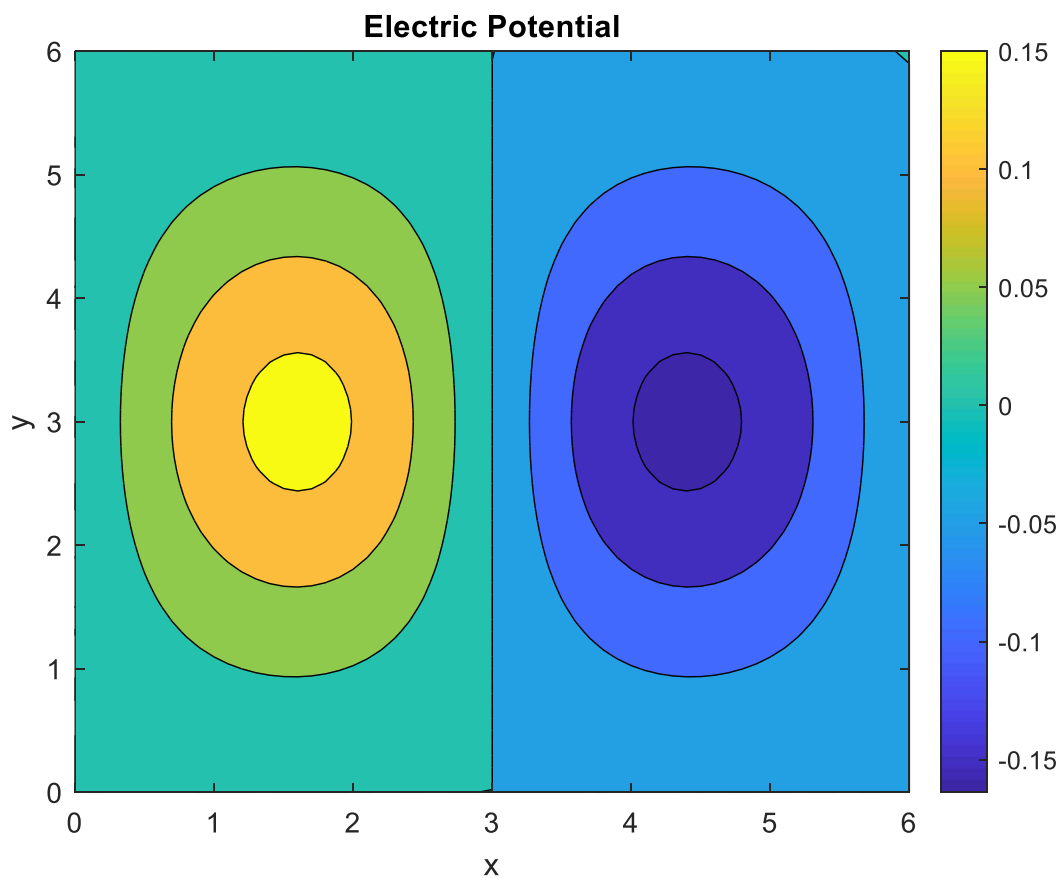


Case 6:

Case of two point charges of equal magnitude and opposite signs located inside the rectangular space with zero electric potential at the boundaries.

Command Window

```
Enter the number of charges: 2
Enter the minimum x-coordinate: 0
Enter the maximum x-coordinate: 6
Enter the minimum y-coordinate: 0
Enter the maximum y-coordinate: 6
Enter charge 1 value: 2
Enter charge 1 x-coordinate: 2
Enter charge 1 y-coordinate: 3
Enter charge 2 value: -2
Enter charge 2 x-coordinate: 4
Enter charge 2 y-coordinate: 3
Enter electric potential at the top boundary: 0
Enter electric potential at the bottom boundary: 0
Enter electric potential at the left boundary: 0
Enter electric potential at the right boundary: 0
```

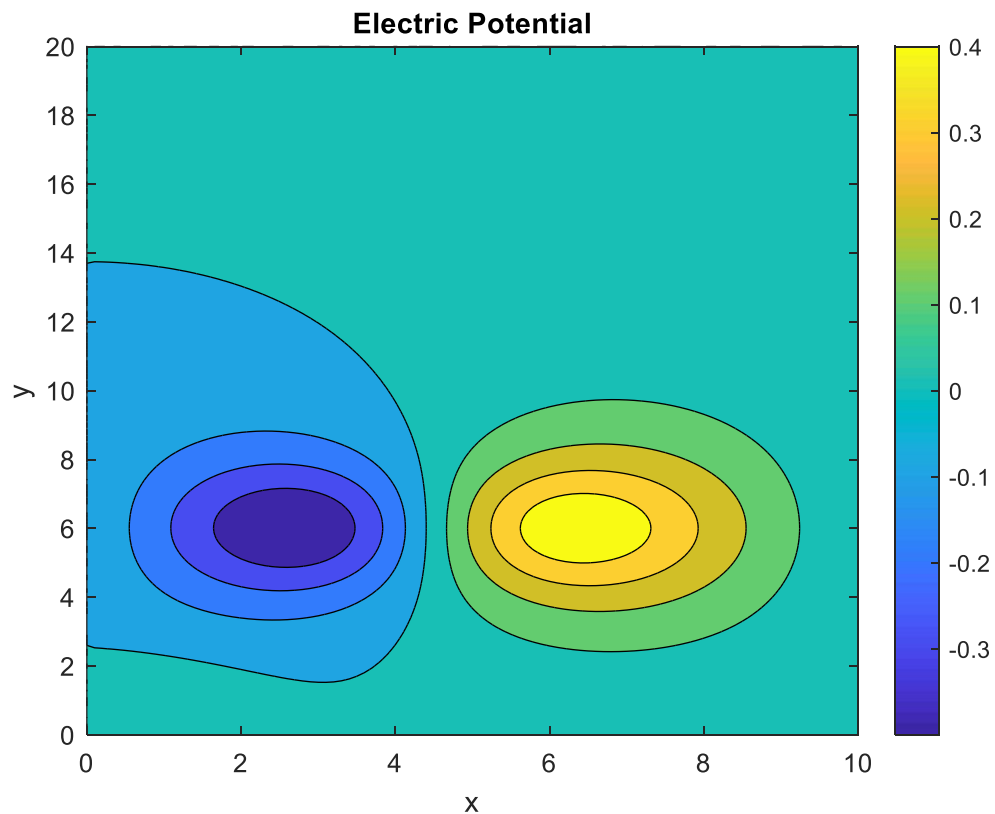


Case 7:

Case of three point charges, two positive and one negative, forming a triangle in the rectangular space with zero electric potential at the boundaries.

Command Window

```
Enter the number of charges: 3
Enter the minimum x-coordinate: 0
Enter the maximum x-coordinate: 10
Enter the minimum y-coordinate: 0
Enter the maximum y-coordinate: 20
Enter charge 1 value: 3
Enter charge 1 x-coordinate: 0
Enter charge 1 y-coordinate: 0
Enter charge 2 value: -3
Enter charge 2 x-coordinate: 3
Enter charge 2 y-coordinate: 6
Enter charge 3 value: 3
Enter charge 3 x-coordinate: 6
Enter charge 3 y-coordinate: 6
Enter electric potential at the top boundary: 0
Enter electric potential at the bottom boundary: 0
Enter electric potential at the left boundary: 0
Enter electric potential at the right boundary: 0
```

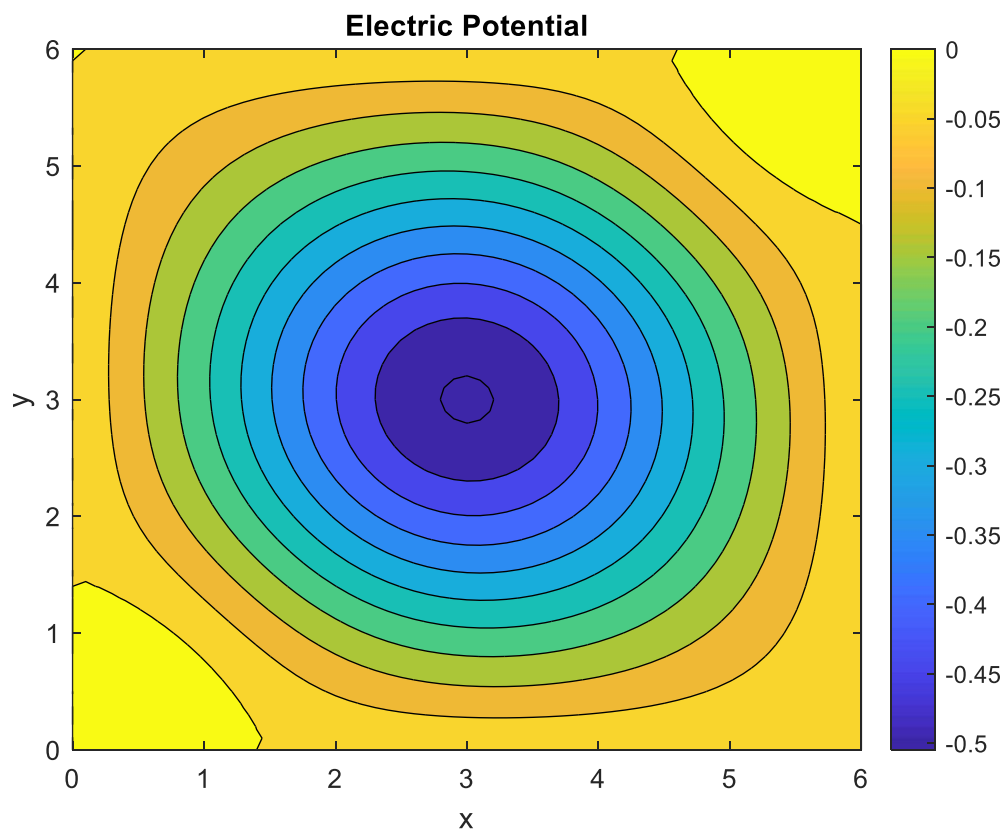


Case 8:

Case of three point charges, two positive on the opposite boundaries and one negative at center, in the rectangular space with zero electric potential at the boundaries.

Command Window

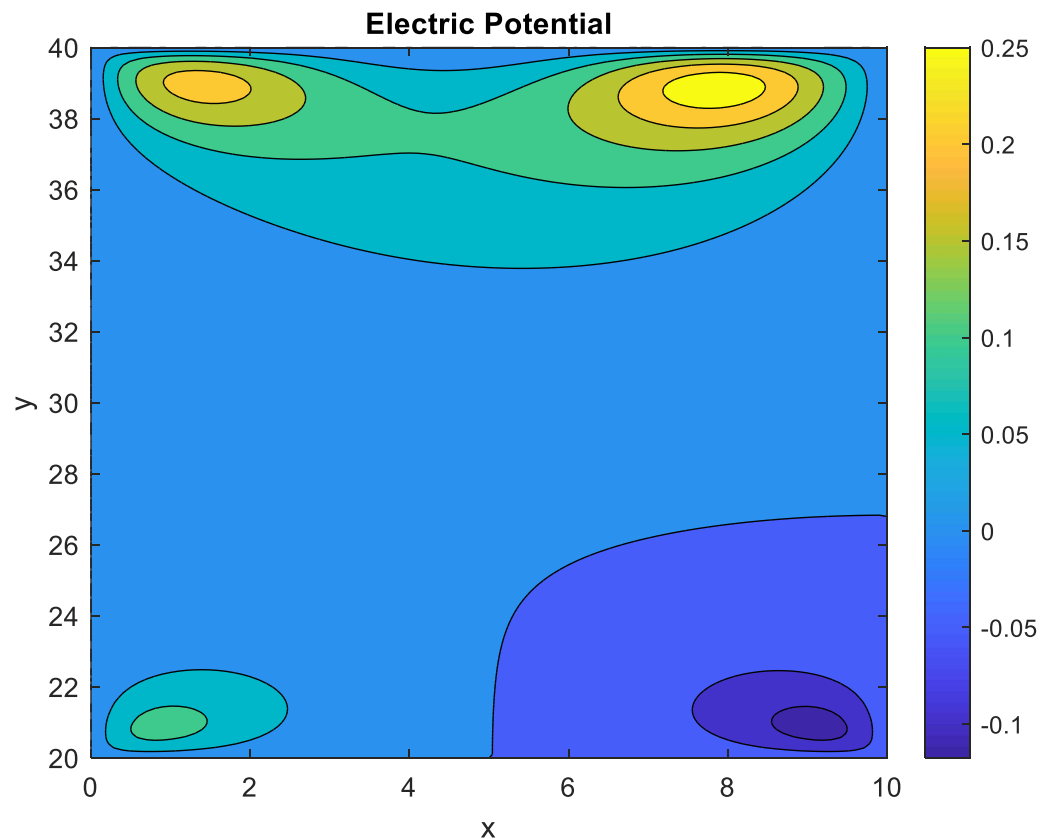
```
Enter the number of charges: 3
Enter the minimum x-coordinate: 0
Enter the maximum x-coordinate: 6
Enter the minimum y-coordinate: 0
Enter the maximum y-coordinate: 6
Enter charge 1 value: 3
Enter charge 1 x-coordinate: 0
Enter charge 1 y-coordinate: 0
Enter charge 2 value: 3
Enter charge 2 x-coordinate: 6
Enter charge 2 y-coordinate: 6
Enter charge 3 value: -3
Enter charge 3 x-coordinate: 3
Enter charge 3 y-coordinate: 3
Enter electric potential at the top boundary: 0
Enter electric potential at the bottom boundary: 0
Enter electric potential at the left boundary: 0
Enter electric potential at the right boundary: 0
```



Case 9:

Case of four point charges, two positive and two negative, forming a trapezoid in the rectangular space with zero electric potential at the boundaries.

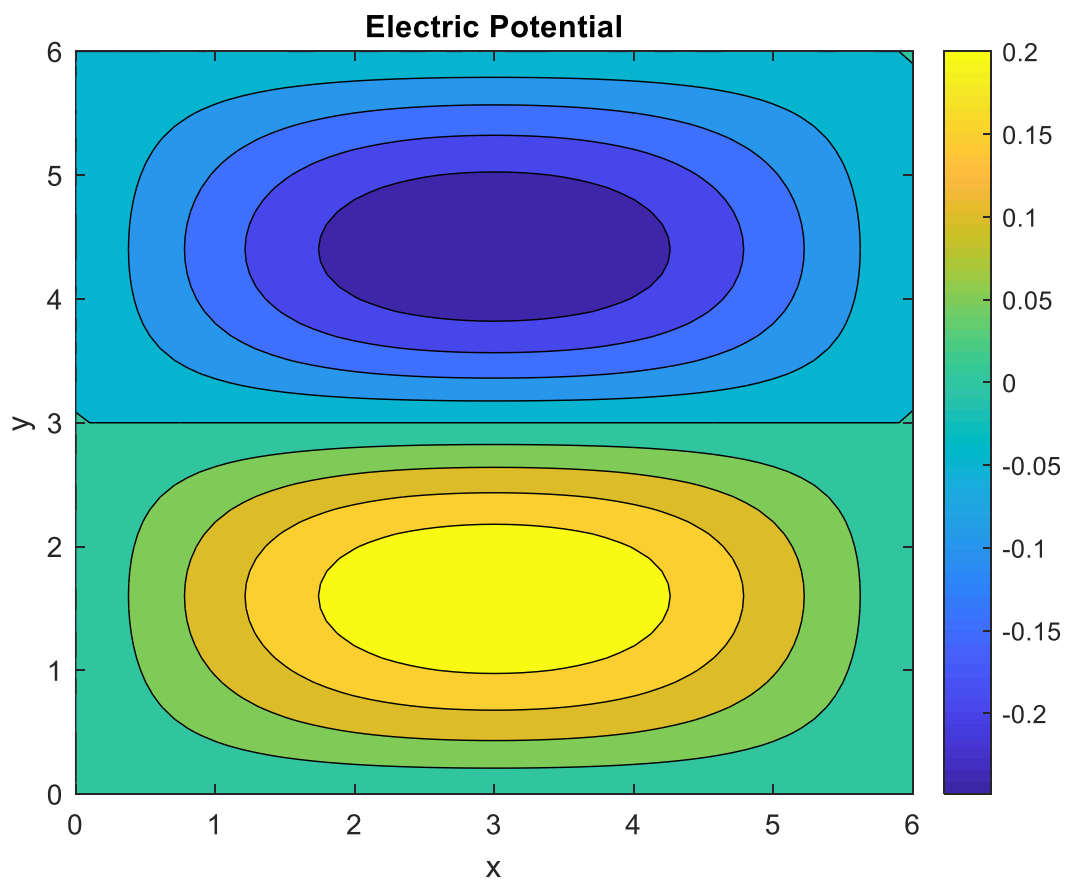
```
Enter the number of charges: 4
Enter the minimum x-coordinate: 0
Enter the maximum x-coordinate: 10
Enter the minimum y-coordinate: 20
Enter the maximum y-coordinate: 40
Enter charge 1 value: 5
Enter charge 1 x-coordinate: 0
Enter charge 1 y-coordinate: 20
Enter charge 2 value: -5
Enter charge 2 x-coordinate: 10
Enter charge 2 y-coordinate: 20
Enter charge 3 value: 5
Enter charge 3 x-coordinate: 1
Enter charge 3 y-coordinate: 40
Enter charge 4 value: 5
Enter charge 4 x-coordinate: 8
Enter charge 4 y-coordinate: 40
Enter electric potential at the top boundary: 0
Enter electric potential at the bottom boundary: 0
Enter electric potential at the left boundary: 0
Enter electric potential at the right boundary: 0
```



Case 10:

Case of four point charges, two positive and two negative, forming a square at the center of the rectangular space with zero electric potential at the boundaries

```
Command Window
Enter the number of charges: 4
Enter the minimum x-coordinate: 0
Enter the maximum x-coordinate: 6
Enter the minimum y-coordinate: 0
Enter the maximum y-coordinate: 6
Enter charge 1 value: 2
Enter charge 1 x-coordinate: 2
Enter charge 1 y-coordinate: 2
Enter charge 2 value: -2
Enter charge 2 x-coordinate: 2
Enter charge 2 y-coordinate: 4
Enter charge 3 value: 2
Enter charge 3 x-coordinate: 4
Enter charge 3 y-coordinate: 2
Enter charge 4 value: -2
Enter charge 4 x-coordinate: 4
Enter charge 4 y-coordinate: 4
Enter electric potential at the top boundary: 0
Enter electric potential at the bottom boundary: 0
Enter electric potential at the left boundary: 0
Enter electric potential at the right boundary: 0
```



Discussion:

The Poisson's equation for electrostatic potential has a great significance in theoretical physics. The Finite Difference Method is important in solving Poisson's equation because it provides an efficient and accurate numerical method for approximating the solution of this second-order partial differential equation. This method approximates the continuous differential equation with a discrete difference equation, allowing for the numerical solution of the equation. This method involves dividing the solution domain into a grid of discrete points and approximating the derivatives of the solution at each point using finite differences.